

《计算机组成原理》实验报告

年级、专业、班级	年级、专业、班级	姓名	姓名
实验题目	实验一简单流水线与运算器实验		
实验时间	YYYY 年 M 月 D 日	实验地点	实验地点
实验成绩	优秀/良好/中等	实验性质	☐ 验证性 ☒ 设计性 ☐ 综合性
教师评价： ☒算法/实验过程正确； ☒源程序/实验内容提交； ☒程序结构/实验步骤合理； ☒实验结果正确； ☒语法、语义正确； ☒报告规范； 其他： 评价教师：钟将			
实验目的 (1)理解流水线设计原理； (2)了解算术逻辑单元 ALU 的原理； (3)熟悉并运用 Verilog 语言设计 ALU； (4)熟悉并运用 Verilog 语言设计流水线全加器；			

报告完成时间: 2026 年 6 月 2 日

1 实验内容

1.1 ALU 设计实验

实验要求实现以下算术运算功能,其对应的控制码及功能如下:

F _{2:0}	功能	F _{2:0}	功能
000	A + B(Unsigned)	100	$\overline{A}$
001	A - B	101	SLT
010	A AND B	110	未使用
011	A OR B	111	未使用

表 1: 算数运算控制码及功能

实验要求:

1. 根据 ALU 原理图,使用 Verilog 语言定义 ALU 模块,其中输入输出端口参考实验原理,运算指令码长度为 [2:0]。
2. 仿真时 B 端口输入为 32h'01, A 端口输入参照 4.1 中表格
3. 实现 SLT 功能。
4. 验证表 1 中所有功能。
5. 给出 RTL 源程序(.v 文件)

1.2 流水线实验

本次实验为仿真实验,设计完成后仅需进行行为仿真。

实验要求:

1. 实现 4 级流水线 8bit 全加器,需带有流水线暂停和刷新;
2. 模拟流水线暂停,仿真时控制 10 周期后暂停流水线 2 周期(第 2 级),流水线恢复流动;
3. 模拟流水线刷新,仿真时控制 15 周期时流水线刷新(第 3 级)。

2 实验设计

2.1 ALU

2.1.1 功能描述

ALU 模块实现 32 位数据的算术和逻辑运算。模块接收两个 32 位操作数 A 和 B,以及 3 位控制码 F[2:0],输出 32 位运算结果 Result。支持 6 种运算:无符号加法、减法、按位与、按位或、按

位取反和小于置位(SLT)。

根据指导书,ALU 有两种实现方式:

- **assign** 实现: 使用连续赋值语句和三目运算符, 适合纯组合逻辑的简单运算
- **always** 实现: 使用 `always@(*)` 块和 `case` 语句, 结构更清晰, 便于扩展

本实验采用 `always` 实现方式, 扩展支持 SLT 运算。

2.1.2 接口定义

表 2: ALU 模块接口定义

信号名	方向	位宽	功能描述
A	Input	32-bit	操作数 A
B	Input	32-bit	操作数 B
F	Input	3-bit	运算控制码, 选择 ALU 执行的运算类型
Result	Output	32-bit	运算结果, 输出至七段数码管显示

2.1.3 逻辑控制

ALU 采用纯组合逻辑实现, 通过 `case` 语句根据控制码 `F[2:0]` 选择对应运算:

F[2:0]	功能	说明
000	A + B (Unsigned)	无符号加法
001	A - B	减法
010	A AND B	按位与
011	A OR B	按位或
100	$\overline{A}$	按位取反
101	SLT	有符号比较, A < B 时输出 1, 否则输出 0
110	未使用	输出 0
111	未使用	输出 0

表 3: ALU 运算控制码及功能

SLT 运算使用 Verilog 内置函数 `$signed()` 将操作数转换为有符号数后进行比较, 按照补码规则判断大小。

2.2 4 级流水线 8bit 全加器

2.2.1 功能描述

实现 4 级流水线 8 位全加器, 参考指导书 Listing 5 的 `adder_8bits_4steps` 模块。将 8 位加法运算拆分为 4 个流水阶段, 每级处理 2 位数据的加法及进位传递。支持流水线暂停和刷新控制, 用于模拟流水线冲突和异常处理。

流水线各级功能划分:

- **第 1 级:** 输入寄存, 计算低 2 位 (`bit[1:0]`) 的加法结果及进位
- **第 2 级:** 计算次低 2 位 (`bit[3:2]`) 的加法, 拼接第 1 级的部分和
- **第 3 级:** 计算次高 2 位 (`bit[5:4]`) 的加法, 拼接前两级的部分和
- **第 4 级:** 计算最高 2 位 (`bit[7:6]`) 的加法, 输出最终 8 位和与进位

2.2.2 接口定义

表 4: 流水线全加器接口定义

信号名	方向	位宽	功能描述
<code>clk</code>	Input	1-bit	时钟信号
<code>rst_n</code>	Input	1-bit	异步复位信号, 低有效
<code>cin_a</code>	Input	8-bit	操作数 A
<code>cin_b</code>	Input	8-bit	操作数 B
<code>c_in</code>	Input	1-bit	进位输入
<code>stall</code>	Input	1-bit	流水线暂停信号, 高有效
<code>flush</code>	Input	1-bit	流水线刷新信号, 高有效
<code>c_out</code>	Output	1-bit	进位输出
<code>sum_out</code>	Output	8-bit	加法结果和

2.2.3 逻辑控制

流水线控制机制:

指导书介绍了流水线的控制逻辑设计方法——基于 `valid/ready` 的握手协议:

- `pipeX_valid`: 流水级 X 中是否有有效数据
- `pipeX_allowin`: 流水级 X 是否允许数据进入
- `pipeX_ready_go`: 流水级 X 是否完成当前数据处理
- `pipeX_to_pipeY_valid`: 从 X 级传递到 Y 级的数据是否有效

其核心思想是将流水线的控制逻辑与数据通路分离。`pipeX_data` 属于数据通路, `pipeX_valid/allowin` 属于控制逻辑。

本实验的简化控制:

1. 正常流动: stall=0 且 flush=0 时, 每个时钟周期数据向下一级推进
2. 暂停: stall=1 时, 所有流水线级寄存器保持当前值不变。用于模拟数据相关或外部等待场景
3. 刷新: flush=1 时, 所有流水线寄存器清零。flush 优先级高于 stall

仿真控制时序:

- 第 10 个时钟周期后, stall 信号有效 2 个周期, 模拟第 2 级流水线暂停, 观察流水线各级寄存器保持情况
- 第 15 个时钟周期时, flush 信号有效 1 个周期, 模拟第 3 级流水线刷新, 观察流水线寄存器清零和重新流动

进位传递机制: 每级流水线使用 {进位, 部分和} 的拼接方式传递数据。c_out_t1 从第 1 级产生, 传递至第 2 级; c_out_t2 传递至第 3 级; c_out_t3 传递至第 4 级。最终进位 c_out 由第 4 级产生, 最终和 sum_out 由 4 级的部分和拼接而成。

3 实验过程记录

3.1 ALU 模块设计与仿真

工作内容:

1. 根据实验要求, 使用 Verilog 语言编写 32 位 ALU 模块, 支持加法、减法、与、或、非、SLT 共 6 种运算
2. 编写 ALU 测试激励, 将 B 端口固定为 32'h01, A 端口依次输入表 5 中的 Num1 值
3. 在 Vivado 中进行行为仿真, 验证 6 种运算功能的正确性
4. 整理仿真波形, 记录验证结果

3.2 流水线全加器设计与仿真

工作内容:

1. 设计 4 级流水线 8 位全加器, 将 8 位加法划分为 4 个流水阶段, 每级处理 2 位
2. 实现 stall 和 flush 控制信号, 支持流水线暂停和刷新功能
3. 编写测试激励, 验证正常加法功能、流水线暂停功能和流水线刷新功能
4. 在仿真中控制第 10 周期后暂停 2 周期, 第 15 周期时刷新
5. 在 Vivado 中进行行为仿真, 整理仿真波形

3.3 问题与解决方案

3.4 问题 1:SLT 运算的符号位处理

问题描述: SLT 指令需要将有符号数进行比较,但 ALU 输入为 32 位无符号类型。直接比较会将所有输入视为无符号数,导致比较结果错误。

解决方案:使用 Verilog 内置函数 \$signed() 将操作数 A 和 B 转换为有符号数后再进行比较,确保 SLT 按照补码规则正确比较大小。当 $A < B$ 时输出 1,否则输出 0。

3.5 问题 2:流水线暂停时数据保持

问题描述:当流水线暂停信号 stall 有效时,需要保持所有流水线级寄存器的值不变。如果在 stall 期间仍有新数据到达输入端,可能导致数据丢失或覆盖。

解决方案:在所有流水线寄存器的 always 块中加入 stall 判断条件。当 stall 有效时,寄存器保持原值不更新;只有 stall 无效时才正常推进数据。

3.6 问题 3:flush 与 stall 的优先级

问题描述:当 flush 和 stall 同时有效时,需要确定哪个信号优先级更高。

解决方案:flush 信号用于处理分支预测失败等异常情况,需要立即清空流水线,因此 flush 优先级高于 stall。在 Verilog 代码中,优先判断 flush 信号。

4 实验结果及分析

4.1 ALU 验证实验结果

ALU 验证实验将 B 端口固定为 32'h01, A 端口依次输入不同的测试值,验证 6 种运算功能的正确性。实验结果如表 5 所示。

操作	F[2:0]	Num1 (A)	Result
A + B (Unsigned)	000	32'h00000002	32'h00000003
A - B	001	32'h000000FF	32'h000000FE
A AND B	010	32'h000000FE	32'h00000000
A OR B	011	32'h000000AA	32'h000000AB
$\overline{A}$	100	32'h000000F0	32'hFFFFFF0F
SLT	101	32'h00000081	32'h00000000

表 5: ALU 验证结果表

结果分析:

1. **A+B (Unsigned):** $2 + 1 = 3$, 结果正确
2. **A - B:** $0xFF - 0x01 = 0xFE$, 结果正确
3. **A AND B:** $0xFE \& 0x01 = 0x00$, 结果正确
4. **A OR B:** $0xAA | 0x01 = 0xAB$, 结果正确
5. **NOT A:** $\sim 0x000000F0 = 0xFFFFF0F$, 结果正确
6. **SLT:** $\$signed(0x00000081) = 129, \$signed(0x00000001) = 1, 129 < 1$ 为假, 输出 0, 结果正确

4.2 ALU 仿真波形图

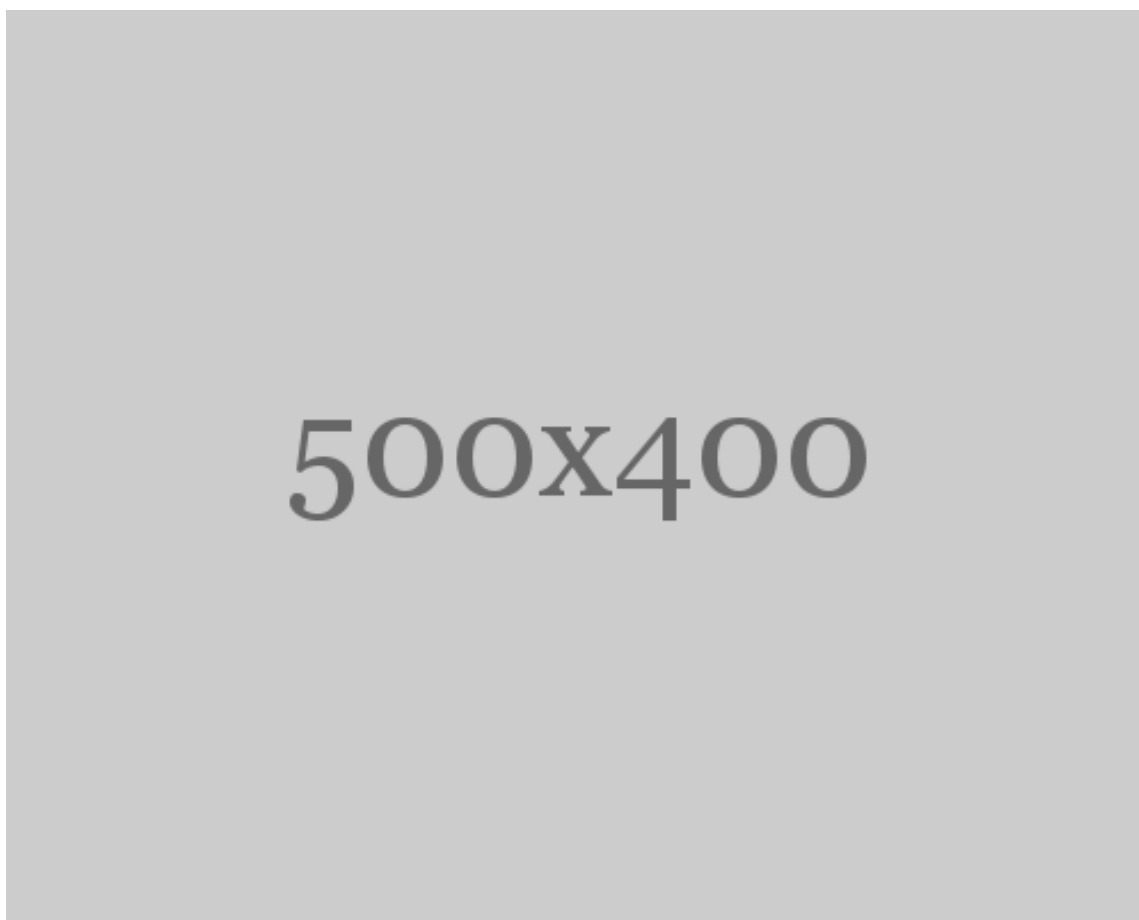


图 1: ALU 行为仿真波形图

4.3 流水线阻塞仿真图

在仿真中控制第 10 个时钟周期后, stall 信号有效 2 个周期, 模拟第 2 级流水线暂停。暂停期间, 流水线各寄存器保持原值不变。2 个周期后 stall 信号无效, 流水线恢复正常流动。



500x400

图 2: 流水线暂停仿真波形图

4.4 流水线刷新仿真图

在仿真中控制第 15 个时钟周期时,flush 信号有效 1 个周期,模拟第 3 级流水线刷新。刷新期间,流水线各级寄存器被清零,之后流水线重新开始流动。

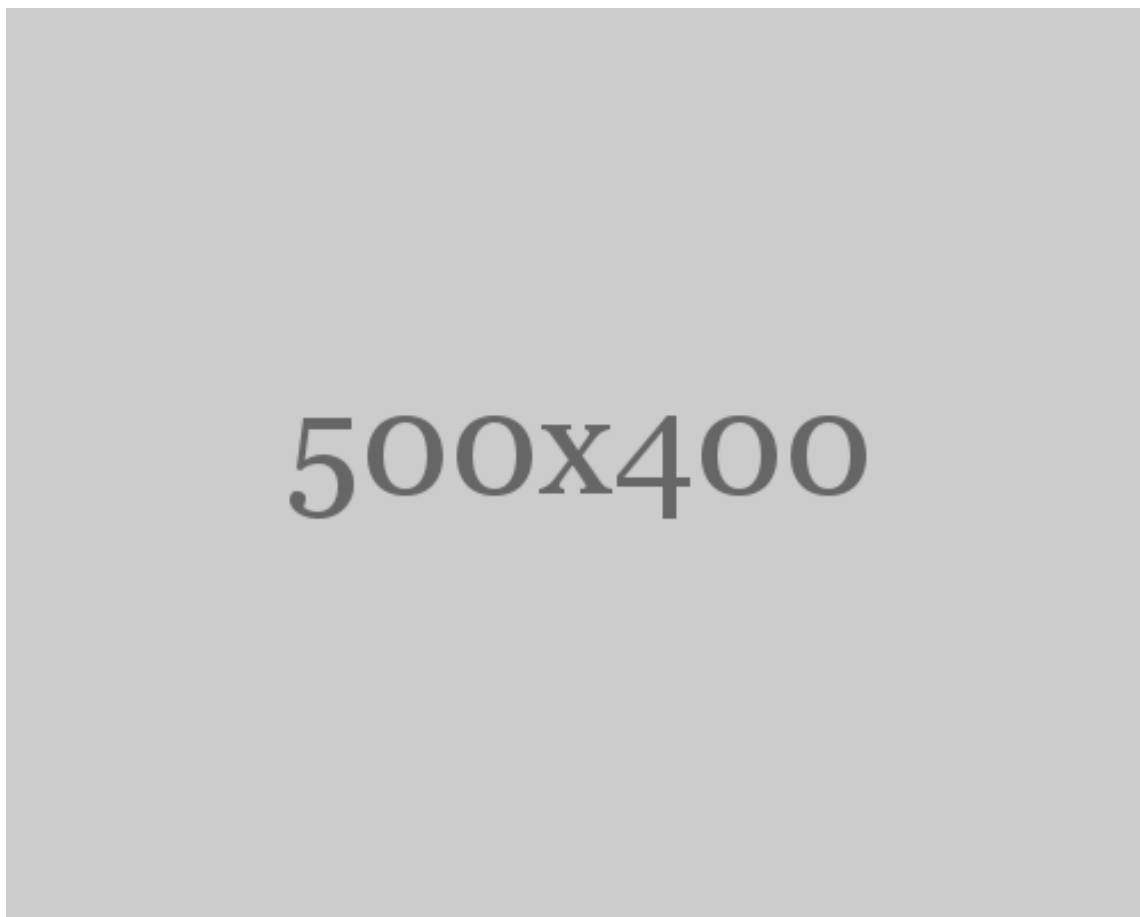


图 3: 流水线刷新仿真波形图

A ALU 代码 (alu.v) ——always 实现

参考指导书 Listing 7 的 always 实现方式,32 位 ALU 模块支持 6 种运算(含 SLT):

```
module alu (  
    input  [31:0] A,  
    input  [31:0] B,  
    input  [2:0]  F,  
    output reg [31:0] Result  
);  
  
    always @(*) begin  
        case (F)  
            3'b000: Result = A + B;  
            3'b001: Result = A - B;  
            3'b010: Result = A & B;
```

```

        3'b011: Result = A | B;
        3'b100: Result = ~A;
        3'b101: Result = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0;
        default: Result = 32'd0;
    endcase
end

endmodule

```

B ALU 代码 (alu_assign.v) ——assign 实现

参考指导书 Listing 6 的 assign 实现方式(不含 SLT, 仅 5 种基本运算):

```

module alu_assign (
    input  wire [7:0]  num1,
    input  wire [2:0]  op,
    output wire [31:0] result
);

    wire [31:0] num2;
    wire [31:0] Sign_extend;

    assign num2 = 32'h00000001;
    assign Sign_extend = {24'b0, num1[7:0]};

    assign result = (op == 3'b000) ? Sign_extend + num2 :
                    (op == 3'b001) ? Sign_extend - num2 :
                    (op == 3'b010) ? Sign_extend & num2 :
                    (op == 3'b011) ? Sign_extend | num2 :
                    (op == 3'b100) ? ~Sign_extend :
                    32'h00000000;

endmodule

```

C 4 级流水线 8bit 全加器代码 (adder_8bits_4steps.v)

参考指导书 Listing 5 的 4 级流水线 8 位加法器, 增加 stall 和 flush 控制:

```

module adder_8bits_4steps (
    input      clk ,
    input      rst_n ,
    input [7:0] cin_a ,
    input [7:0] cin_b ,
    input      c_in ,
    input      stall ,

```

```

input          flush ,
output         c_out ,
output [7:0]   sum_out
);

reg           c_out;
reg          c_out_t1, c_out_t2, c_out_t3;
reg [7:0]    sum_out;
reg [1:0]    sum_out_t1;
reg [3:0]    sum_out_t2;
reg [5:0]    sum_out_t3;

// Stage 1: bits[1:0]
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        {c_out_t1, sum_out_t1} <= 3'd0;
    end else if (flush) begin
        {c_out_t1, sum_out_t1} <= 3'd0;
    end else if (!stall) begin
        {c_out_t1, sum_out_t1} <= {1'b0, cin_a[1:0]} + {1'b0, cin_b[1:0]} +
            c_in;
    end
end

// Stage 2: bits[3:2]
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        {c_out_t2, sum_out_t2} <= 5'd0;
    end else if (flush) begin
        {c_out_t2, sum_out_t2} <= 5'd0;
    end else if (!stall) begin
        {c_out_t2, sum_out_t2} <= {{1'b0, cin_a[3:2]} + {1'b0, cin_b[3:2]} +
            c_out_t1, sum_out_t1};
    end
end

// Stage 3: bits[5:4]
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        {c_out_t3, sum_out_t3} <= 7'd0;
    end else if (flush) begin
        {c_out_t3, sum_out_t3} <= 7'd0;
    end else if (!stall) begin
        {c_out_t3, sum_out_t3} <= {{1'b0, cin_a[5:4]} + {1'b0, cin_b[5:4]} +
            c_out_t2, sum_out_t2};
    end
end

// Stage 4: bits[7:6], final output

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        {c_out, sum_out} <= 9'd0;
    end else if (!stall) begin
        {c_out, sum_out} <= {{1'b0, cin_a[7:6]} + {1'b0, cin_b[7:6]} + c_out_t3
            , sum_out_t3};
    end
end
endmodule

```

D 流水线全加器测试激励代码 (tb_pipeline_adder.v)

```

module tb_pipeline_adder;
    reg        clk;
    reg        rst_n;
    reg  [7:0] cin_a;
    reg  [7:0] cin_b;
    reg        c_in;
    reg        stall;
    reg        flush;
    wire       c_out;
    wire  [7:0] sum_out;

    integer cycle_count;

    adder_8bits_4steps uut (
        .clk(clk),
        .rst_n(rst_n),
        .cin_a(cin_a),
        .cin_b(cin_b),
        .c_in(c_in),
        .stall(stall),
        .flush(flush),
        .c_out(c_out),
        .sum_out(sum_out)
    );

    // 100MHz clock: T = 10ns
    always #5 clk = ~clk;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            cycle_count <= 0;
        else
            cycle_count <= cycle_count + 1;
    end
end

```

```

initial begin
    clk    = 0;
    rst_n = 0;
    cin_a = 8'd0;
    cin_b = 8'd0;
    c_in  = 1'b0;
    stall = 1'b0;
    flush = 1'b0;

    #20 rst_n = 1;
    #10;

    // Normal operation: 0x12 + 0x34 = 0x46
    cin_a = 8'h12;
    cin_b = 8'h34;

    // Cycle 10: stall for 2 cycles (stage 2)
    wait(cycle_count >= 10);
    stall = 1'b1;
    repeat(2) @(posedge clk);
    stall = 1'b0;

    // Cycle 15: flush (stage 3)
    wait(cycle_count >= 15);
    flush = 1'b1;
    @(posedge clk);
    flush = 1'b0;

    #100;
    $finish;
end
endmodule

```